SVG Maps with D3

Visualisation and Sensing BSC2 2025 | Tom Armitage | t.armitage@arts.ac.uk

In this exercise, you'll use D3 to draw SVG maps and data visualisations in the browser.

Always prefer typing out code, rather than copy-pasting - unless the text tells you otherwise.

Timing

This workshop is designed to take an hour, including lecturer walkthrough.

Requirements

VS Code with Live Server extension

Getting Started

Open this directory in Visual Studio Code, and run VS Code Live Server. You must access your code through a server running at localhost - you can't just double-click `index.html`.

For each exercise, I suggest starting with a new file. Start by basing your work on `index.html`

1. Up and running with D3

You should work in `index.html`, *or* in a copy of it in the same directory.

Start Live Server running in this directory.

At `http://localhost:5500` (or `http://localhost:5500/your_page.html`, if you've copied the index file to `your_page.html`), you should see a white page with the title "Maps Workshop".

> **Notes on the template HTML**
>
> The `index.html` template contains:
>
> - appropriate code to load the D3 library.
> - very simple default CSS

Setting up the D3 visualisation.

Task: Start by adding an SVG element to the page. We'll be adding all our path data to this D3 selection:

```
const vizWidth = 1000;
const vizHeight = 500;

const svg = d3
  .select("#container")
  .append("svg")
  .attr("width", vizWidth)
  .attr("height", vizHeight);
```

Task: Append a group to this. We'll put our map data in this group. `svg` is already a D3 selection; you can `append` a `g` element to it; store this in a constant called `map`.

When you've done that, we're nearly ready to append geodata. But we need two more things.

Projections and PathGenerators

We need to take any lat/lng data and use a **projection** to project it into our flat X/Y co-ordinate space.

D3 has many geographic projections. Create an *equirectangular* one:

```
const projection = d3.geoEquirectangular();
```

`projection` will be a *function* that takes a point (of the form `[lng,lat]`) and return an X-Y position in the D3 space.

> D3 always handles co-ordinates in the order (longitude, latitude)!

You should also create a *path generator*:

```
const pathGenerator = d3.geoPath(projection);
```

Here's the documentation for `d3.geoPath`. It tells us that `d3.geoPath` takes a projection, and returns a function that will generate the SVG path data - what goes in the `d` attribute of an SVG `patha` - for a GeoJSON `LineString`. That means: given a GeoJSON path, it'll return an SVG path, projected into our projection, and within the visualisation co-ordinate system.

With a projection and pathgenerator, you're ready to load GeoJSON.

Loading and drawing GeoJSON.

Load the GeoJSON in `/data/countries.geojson` with the Fetch API.

```
fetch("/data/countries.geojson").then((response) => {
  // response is a Response object
  response.json().then((data) => {
    // data is our geoJSON object.
    //
    // now: append something to the map based on this!
  });
});
```

> **Fetch**
>
> The Fetch API is a Javascript interface inside modern browsers for making HTTP requests and parsing the response.
>
> Making a GET request is simple - `fetch(url)`. `fetch` can also take an options object, which will allow you to use other HTTP verbs, pass additional headers, etc.
>
> The fetch API returns its data as a Promise. There are two ways to deal with promises: using `async/await`, and using `then/catch` methods. We'll use the latter, as it might appear more familiar for now.

Have a look at the contents of `/data/countries.geojson`. You'll see that it's a FeatureCollection, and each Feature in this collection is a country - some data about it, and the Geometry of its outline.

Task: Draw a path for each item in `data.features`.

In D3 terms, that means:

- `selectAll` the paths in `map` (which, remember, don't exist *yet*)
- bind their `data` to `data.features`
- on `join`, make a `path` object
- set the `d` attr to whatever happens when you pass the feature to the pathGenerator

(`.attr('d',pathGenerator)`)
- set the `stroke` attribute to something like a shade of grey (say, `#ccc`)
- set the `fill` attribute to a light shade, eg `#eee` (for now)

Refer to the D3 documentation, to refresh your memory on D3 selections (or: the D3 exercise from a few weeks back). The above should get you to seeing a map of the countries of the world.

Explore the SVG element using the Developer Console in the browser inspector to see what has been generated.

Adding behaviour

Let's use interaction to add some of the data from the GeoJSON file into our document.

Calling the `on` method on a D3 selection can attach behaviour to each item in the selection. The method signature looks like this:

```
yourD3Selection.on("eventname", (event, datum) => {
  /* do something */
});
```

When `eventname` happens, the second parameter - the function you are passing in - will be called; the original `event` as well as the related piece of data (the `datum`) are available to the function.

Task: On `mouseover` for your selection of paths, set the `innerText` of the div with id "country" to the name of the country. `datum.properties.NAME_EN` will contain the name of the country. (Why? What other fields could you make appear?).

Task: On `mouseout` , clear the field.

Choropleth maps

Task: Create an ordinal color scale (create this outside all loops/blocks, near where you created your projection in the code):

```
const colorScale = d3.scaleOrdinal(d3.schemeCategory10);
```

(There are lots of other ways to pick color scales in D3; this will do for now. You may wish to investigate other ways of picking colours for your work)

Task: Color each country according to its `ECONOMY` property

The ECONOMY field on properties groups countries into different economic groups - from G7 nations down to developing nations.

Remember that attribute in D3 can be supplied a function, to calculate them based on the value. For instance,
`.attr('fill', (d,i) => { /* return value for fill based on data or index */})`.

Fill each country according to the ECONOMY property. This has two steps:

- convert the economy property into a number (ie, which category is it in, from 0-6)
- pass that number into the chromatic scale.

I'll share how to do the former for you.

Straight after parsing the JSON from your request - let's say I called the JSON `data`:

```
// get the ECONOMY field for every item
const economies = data.features.map((d) => d.properties.ECONOMY);
// make a new Set of those economies. Sets contain unique items, so any economie
const uniqueEconomies = [...new Set(economies)].sort();
```

You can `console.log(uniqueEconomies)` if you want to see what the results are.

To set the colour of the country, you need to pick a color from the colorscale according to the value of its economy property. Each different economy type should have a different colour; countries with the same economy type will be the same colour.

In code, that means you need to *find the index of the country's economy in the* `uniqueEconomies` *variable*. You can find the index of an item in an array with `array.indexOf(item)`

Remember that D3 attributes can be passed a function, to calculate an attribute value for each item in the dataset.

Task: Finally, add the economy value to the text that appears on Mouseover. Consider formatting it with HTML.

Stretch Goal: Meteorites

If you get this far: have a look at `meteorites.json`. Can you make a second Fetch request to get this data, and plot meteorite falls as circles on your map?